Common requirements:
Your program should be user friendly and easy to use. You have to make an object oriented implementation, but it is not necessary to use multilayer architectures (MV, MVC etc.). You have to use simple graphics for the game display. The "sprite" of the player's character should be able to moved with the well known WASD keyboard buttons. You can also implement mouse event handlers to other functions of the game. You can generate the game levels with an algorithm, or simply load them from files. If you load the levels from file, then each one should be put into different files, and you have to create at least 10 predefined levels. If you generate the levels, then take care that the levels should be playable (player should be able to solve it). Each game needs to have a timer, which counts the elapsed time since the start of the game level. The documentation should contain the description of the task, its analysis, the structure of the program (UML class diagram), a chapter for the implementation, which describes the most interesting algorithms (e.g. that generates the level) etc. of the program. Also, don't forget to show the connections between the events and their handlers. The task description should contain the minimal requirements. It is free to add new functionalities to the games.

Exercies: 1. Yogi Bear Yogi Bear wants to collect all the picnic baskets in the forest of the Yellowstone National Park. This park contains mountains and trees, that are obstacles for Yogi. Besided the obstacles, there are rangers, who make it harder for Yogi to collect the baskets. Rangers can move only horizontally or vertically in the park. If a ranger gets too close (one unit distance) to Yogi, then Yogi loses one life. (It is up to you to define the unit, but it should be at least that wide, as the sprite of Yogi.) If Yogi still has at least one life from the original three, then he spawns at the entrance of the park. During the adventures of Yogi, the game counts the number of picnic baskets, that Yogi collected. If all the baskets are collected, then load a new game level, or generate one. If Yogi loses all his lives, then show a popup messagebox, where the player can type his name and save it to the database. Create a menu item, which displays a highscore table of the players for the 10 best scores. Also, create a menu item which restarts the game.

1. Task Description

Game Overview

Yogi Bear must collect all picnic baskets in Yellowstone National Park while avoiding park rangers and navigating around obstacles (trees and mountains).

Game Rules

- **Objective**: Collect all picnic baskets in each level
- **Controls**: WASD keys to move Yogi
- **Lives**: Player starts with 3 lives
- **Enemies**: Rangers patrol horizontally or vertically
- **Danger**: If a ranger gets within one unit (35 pixels) of Yogi, he loses a life
- **Death**: When all lives are lost, the game ends
- **Respawn**: After losing a life, Yogi respawns at the level entrance
- **Progression**: When all baskets are collected, the next level loads
- **Levels**: 10 predefined levels that cycle

Features

1. Timer showing elapsed time since level start
2. Lives counter
3. Basket collection counter
4. Level number display
5. High score system (top 10)
6. Menu with New Game, High Scores, and Exit options

2.Analysis

Game Requirements Analysis

Functional Requirements:

- Player movement using WASD keys
- Collision detection between player and obstacles
- Collision detection between player and baskets
- Proximity detection between player and rangers
- Level loading from files
- Timer counting elapsed time
- Lives management system
- Score tracking and high score table
- Menu system for game control

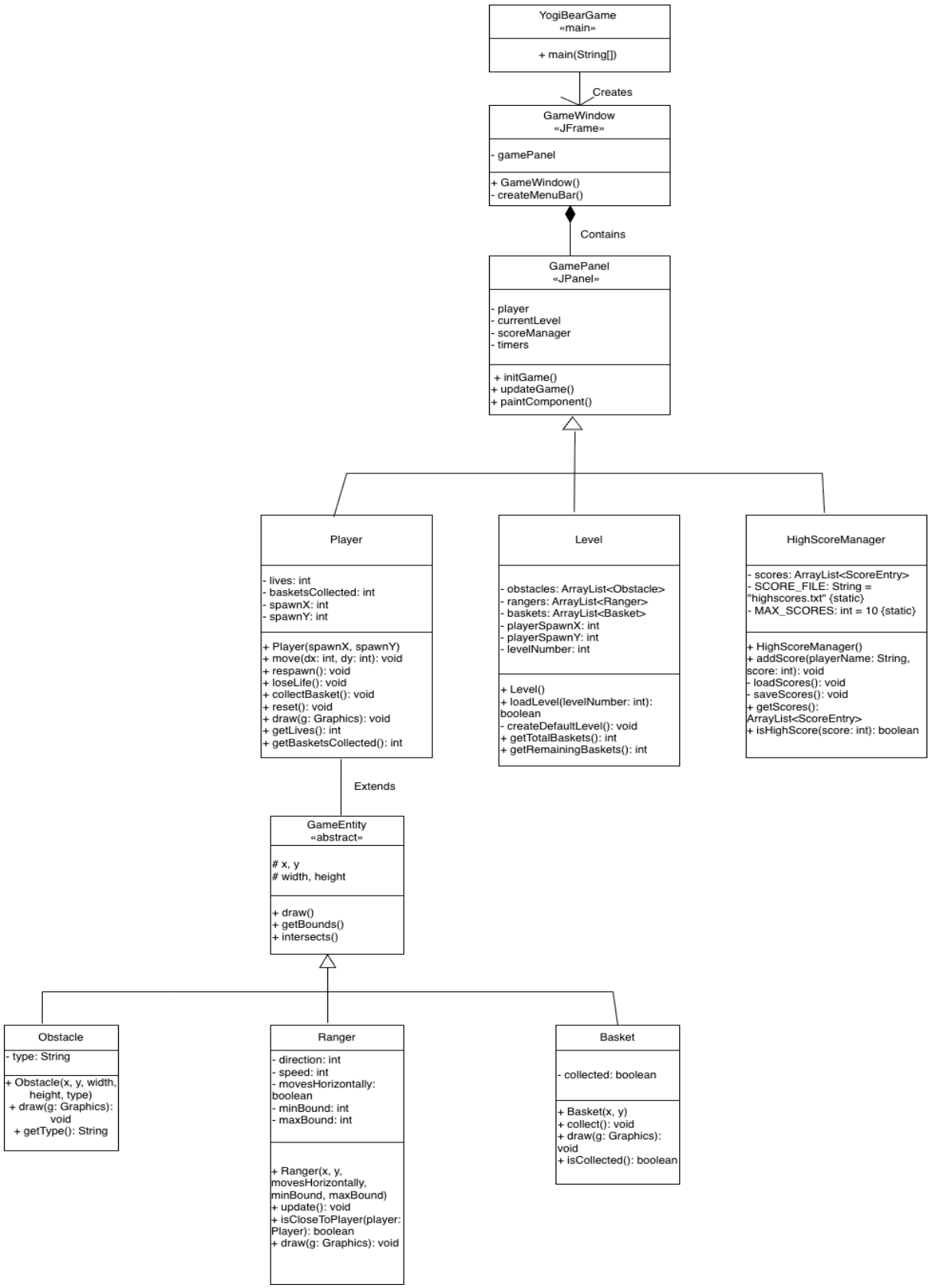
Non-Functional Requirements:

- Simple, clear graphics
- User-friendly interface
- Responsive controls
- Easy to understand gameplay
- Proper file I/O error handling

Game Entities

1. **Player (Yogi Bear)**: Controlled by user, can move in 4 directions
2. **Rangers**: Enemy entities that patrol fixed paths
3. **Baskets**: Collectible items that increase score
4. **Obstacles**: Static barriers (trees and mountains)
5. **Level**: Container for all game entities

3.UML Diagram



4. Implementation Details

Key Algorithms

4.1 Collision Detection

java

```
// Bounding box collision

public boolean intersects(GameEntity other) {

   return this.getBounds().intersects(other.getBounds());
            }
```

This uses Java's Rectangle class to determine if two entities overlap.

4.2 Ranger Patrol Algorithm

java

```
public void update() {

   if (movesHorizontally) {

      x += direction * speed;

      if (x <= minBound || x >= maxBound) {

         direction *= -1;  // Reverse direction

      }

   } else {

      // Similar for vertical movement

   }
            }
```

Rangers patrol back and forth between defined boundaries.

4.3 Proximity Detection

java

```java
public boolean isCloseToPlayer(Player player) {

    int dx = Math.abs(this.x - player.getX());

    int dy = Math.abs(this.y - player.getY());

    int distance = (int) Math.sqrt(dx * dx + dy * dy);

    return distance <= 35;
}
```

Uses Euclidean distance formula to detect when ranger is close to player.

4.4 Level Loading

java

```java
public boolean loadLevel(int levelNumber) {

    // Read file line by line

    // Parse entity types and parameters

    // Create appropriate objects

    // Handle file not found with default level
}
```

File format:

spawn_x spawn_y

type x y [parameters...]

4.5 High Score Management

- Scores are stored in a sorted ArrayList
- When adding a new score, the list is re-sorted
- Only top 10 scores are kept
- Scores persist to file using simple CSV format

Events and their Handlers

Keyboard Events:

KeyPressed → sets corresponding boolean (wPressed, aPressed, etc.)
KeyReleased → clears corresponding boolean

Game Loop → checks booleans and moves player accordingly

Menu Events:

New Game → confirms with user → calls restartGame()
High Scores → calls showHighScores()

Exit → confirms with user → System.exit(0)

Timer Events:

gameTimer (20ms) → updateGame() → repaint()

clockTimer (1000ms) → elapsedSeconds++

Game Loop

Every 20ms:
1. Read keyboard state (WASD)
2. Move player based on input
3. Check boundary collisions
4. Check obstacle collisions (undo movement if collision)
5. Check basket collections
6. Update all rangers
7. Check ranger proximity to player
8. Repaint screen

Test Cases (Load Levels from files)

10 predefined levels

Test Case1 to 10

Level1.txt
50 50
tree 200 150 40 40

```
tree 300 350 40 40
mountain 400 200 60 50
ranger 150 300 h 100 500
basket 550 100
basket 200 400
basket 450 300
```

Level2.txt

```
50 400
tree 250 100 40 40
tree 400 300 40 40
tree 150 250 40 40
mountain 500 150 60 50
mountain 350 450 60 50
ranger 200 200 v 100 450
ranger 450 250 h 200 650
basket 700 50
basket 100 150
basket 600 500
basket 350 200
```

Level3.txt

```
400 50
tree 100 200 40 40
tree 300 400 40 40
tree 600 250 40 40
mountain 200 100 60 50
mountain 500 400 60 50
ranger 300 150 h 150 600
ranger 400 350 v 150 500
ranger 150 450 h 50 400
basket 50 500
basket 700 100
basket 400 250
basket 250 350
basket 600 450
```

Level4.txt

```
50 550
tree 150 100 40 40
tree 350 150 40 40
tree 550 100 40 40
tree 200 350 40 40
tree 500 400 40 40
mountain 300 250 60 50
mountain 600 300 60 50
ranger 250 200 v 100 450
ranger 450 300 h 200 700
ranger 100 450 h 50 350
basket 700 50
```

```
basket 100 100
basket 400 450
basket 650 500
```

Level5.txt

```
400 300
tree 100 100 40 40
tree 700 100 40 40
tree 100 500 40 40
tree 700 500 40 40
tree 400 150 40 40
mountain 250 250 60 50
mountain 550 350 60 50
mountain 350 450 60 50
ranger 200 150 h 100 600
ranger 500 250 v 100 500
ranger 300 400 h 150 650
ranger 600 450 v 200 550
basket 50 50
basket 750 50
basket 50 550
basket 750 550
basket 400 300
```

Level6.txt

```
50 300
tree 200 100 40 40
tree 400 100 40 40
tree 600 100 40 40
tree 200 450 40 40
tree 400 450 40 40
tree 600 450 40 40
mountain 150 250 60 50
mountain 450 250 60 50
mountain 650 350 60 50
ranger 300 200 v 100 500
ranger 500 200 v 100 500
ranger 350 350 h 200 600
basket 100 150
basket 550 150
basket 700 250
basket 150 500
basket 500 500
basket 700 500
```

Level7.txt

```
750 550
tree 100 50 40 40
tree 300 150 40 40
tree 500 250 40 40
```

```
tree 200 350 40 40
tree 400 450 40 40
tree 650 400 40 40
mountain 150 200 60 50
mountain 400 100 60 50
mountain 550 450 60 50
ranger 200 100 h 100 500
ranger 350 250 v 100 450
ranger 500 350 h 300 700
ranger 150 500 v 300 550
basket 50 100
basket 600 100
basket 300 300
basket 650 250
basket 100 500
basket 500 550
basket 700 500
```

Level8.txt

```
400 50
tree 150 150 40 40
tree 350 150 40 40
tree 550 150 40 40
tree 650 150 40 40
tree 250 350 40 40
tree 450 350 40 40
tree 150 500 40 40
tree 550 500 40 40
mountain 300 250 60 50
mountain 500 250 60 50
mountain 400 450 60 50
ranger 200 250 h 100 700
ranger 400 200 v 100 500
ranger 600 350 v 150 550
ranger 300 500 h 150 550
basket 50 200
basket 750 200
basket 100 400
basket 700 400
basket 350 500
basket 500 100
```

Level9.txt

```
50 50
tree 200 200 40 40
tree 400 200 40 40
tree 600 200 40 40
tree 200 400 40 40
tree 400 400 40 40
tree 600 400 40 40
```

mountain 300 100 60 50
mountain 500 100 60 50
mountain 300 500 60 50
mountain 500 500 60 50
ranger 250 150 h 150 650
ranger 450 150 h 250 750
ranger 250 450 h 150 650
ranger 450 450 h 250 750
ranger 350 300 v 150 450
basket 100 300
basket 700 300
basket 400 100
basket 400 500
basket 150 150
basket 650 150
basket 150 450
basket 650 450

Level10.txt
400 550
tree 100 100 40 40
tree 250 100 40 40
tree 400 100 40 40
tree 550 100 40 40
tree 700 100 40 40
tree 150 250 40 40
tree 350 250 40 40
tree 550 250 40 40
tree 650 250 40 40
tree 100 400 40 40
tree 300 400 40 40
tree 500 400 40 40
tree 700 400 40 40
mountain 200 500 60 50
mountain 400 500 60 50
mountain 600 500 60 50
ranger 175 150 h 100 700
ranger 275 200 v 100 500
ranger 475 200 v 100 500
ranger 575 300 h 300 750
ranger 200 450 h 100 600
ranger 500 450 v 250 550
basket 50 50
basket 750 50
basket 50 300
basket 750 300
basket 50 550
basket 750 550
basket 400 300
basket 250 500

basket 550 500
basket 400 450